

Aarhus University

Project number - 2026F25

Parallax - Process Report

Students:

Thea Teresa Lindgaard (202204350)
Niels Jakob Østerberg (202105879)
Kasper Stecher (202100451)
Kasper Bentsen (202103249)

Counsellor:

Lukas Esterle

Aarhus, May 29th 2026

Contents

1. Project Foundation	1
1.1. Project Scope and Goals	1
1.2. Group Formation and Collaboration	1
1.2.1. Roles and Responsibilities	2
1.2.2. Group Contract and Expectations	2
2. Development Methodology	3
2.1. Agile Development with Scrum	3
2.2. Communication and Planning	3
2.2.1. Jira	3
2.2.2. Discord	4
3. Implementation Methodology	5
3.1. Feature-Based	5
3.2. Proof-of-Concept Development	5
3.3. Reflection on Implementation Methodology	5
4. Git Workflow	6
4.1. Repository setup	6
4.2. Branch strategy	6
4.3. Pull requests and code review	7
4.4. Conflict handling	7
4.5. Commit practices	7
4.6. Reflection on Git workflow	7
5. Discussion	8
5.1. What Worked Well	8
5.2. What Didn't Work Well	8
5.3. Realizations	9
6. Conclusion	10
Bibliography	11

1. Project Foundation

The following sections describe the foundation of the project, with a focus on project goals, group collaboration and expectations.

1.1. Project Scope and Goals

The goal for this project was to develop a game that revolved around player cooperation and interdependence through asymmetric roles. The scope was limited to a playable prototype. This allowed the group to focus on the core mechanics, rather than spend large amounts of time producing polished content. Instead, the group focused on puzzle interaction, asymmetric gameplay and player experience. The prototype was intended to test whether the game concept was engaging for players, and technically feasible.

Additionally, restricting the project to a prototype was better suited for this bachelor project, given the time constraint. A prototype scope reduced the risk of unfinished systems, allowed realistic planning, and improved the likelihood of producing a stable final product.

1.2. Group Formation and Collaboration

The group consists of people who worked together for a large part of their education. This was beneficial, because the group knew in advance how they work together best, and have pre-established communication habits.

All group members have completed the Computer Game Technologies course, providing experience in game development prior to the bachelor project. Two group members have multiple years of experience with game development in Unity in addition to the aforementioned course. The combination of group members with a lot of experience, and group members with relatively little experience proved to be a very beneficial balance. It meant that new technologies and innovative ideas were more likely to be considered by those with less Unity experience, while those with more experience could provide quick technical guidance, identify potential issues early, and support efficient implementation decisions.

1.2.1. Roles and Responsibilities

The group did not operate with strict roles during development. Instead, responsibilities were distributed based on personal competence, interest and current project needs. This allowed the group to adapt as the project progressed. Group members contributed to multiple areas of the project, based on project needs.

1.2.2. Group Contract and Expectations

Early in the process, the group created and signed a group contract. The contract outlined basic guidelines for how the group should collaborate, including basic expectations about attendance and participation, communication, and conflict resolution. This contract was intended to assist in the event of unexpected disagreements or collaboration issues during the project. The contraction was never formally needed during the project, as the group did not experience any collaboration issues.

2. Development Methodology

The following sections describe the methodology used to structure development during the project, including tools used for planning and communication.

2.1. Agile Development with Scrum

The group used a modified Scrum[1] methodology to structure the development process. Scrum was chosen because the group had prior experience with the methodology, and because the project involved mechanics and technologies that were new to the team. This made Scrum beneficial, because it allowed the group to work iteratively, enabling experimenting with ideas and adapting continuously as the project evolved.

As previously stated, the group did not employ strict roles. This also applies to the Scrum methodology. The role of Scrum master was only delegated in meetings ad hoc if the group found it to be necessary. This suited the group structure better, as the relatively small group size and collaborative working style made Scrum roles less relevant. Instead, responsibilities were shared across all team members.

Development was organized into 1-week sprints, with one or two stand-up meetings per week, depending on what the group needed. The stand-up meetings ensured that the group was always informed about each other's progress on their respective tasks, and provided opportunities for group members to seek help and offer support. Brief notations about current tasks and progress were made in a separate document, providing a way to track progress, discoveries and potential blockers.

Occasionally, meetings would evolve into collaborative work sessions. These sessions typically involved integration testing, testing multiplayer systems over multiple devices, peer programming or bug-fixing.

2.2. Communication and Planning

The group used different tools to plan and manage the project during development. The choice of tool used during development depended on project needs.

2.2.1. Jira

Jira[2] was primarily used when tasks were clear and stable. In the beginning of development, Jira was used to track tasks in detail, with concrete subtasks and progress bars, as well as assigning due dates to certain tasks. Similarly, Jira was used more heavily towards the end of the project, where tasks again were more clearly defined.

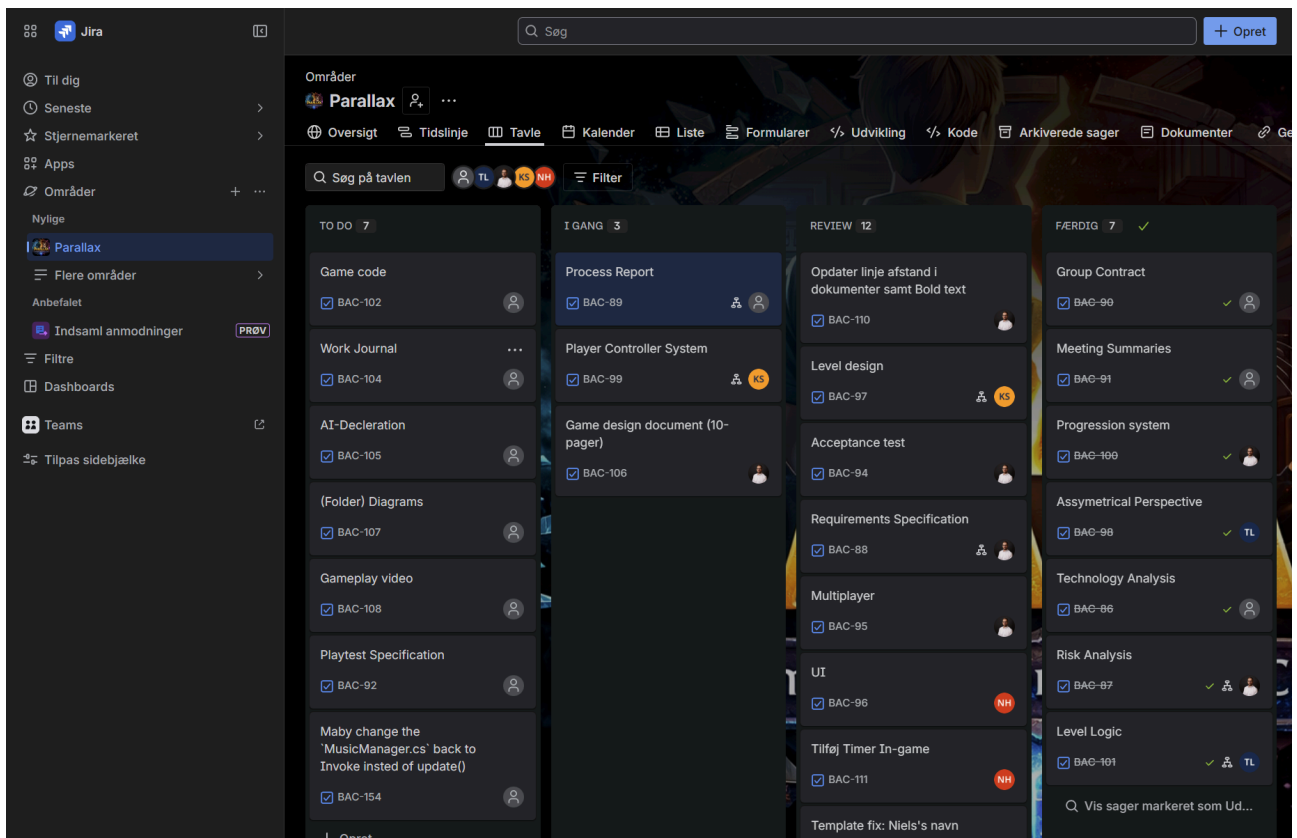


Figure 1: Screenshot from Jira showing how tasks were created and tracked in Jira

Jira was a helpful tool when tasks were known and did not change a lot, because it provided detailed insight into progress made, helped manage which tasks were pending, ongoing, under review, and which were complete. Additionally, Jira allowed team members to make comments on tasks as part of the review process, providing a structured way to track and discuss potential changes. **Figure 1** illustrates how tasks were tracked across these categories.

However, when tasks were rapidly evolving, the overhead of managing Jira became more trouble than it was worth. In the middle of development, tasks became less defined, required more research and experimentation, and it was more difficult to estimate how long tasks would take to complete. During this phase of development, the group switched to Discord.

2.2.2. Discord

Discord[3] was used as the group's primary tool for internal communication. Groups held online meetings and discussions in Discord, as well as sought assistance or asked questions.

Towards the middle of the project, the group began to use Discord as a tool for tracking tasks, in addition to communication. This was useful, because tasks became less clearly defined. More general task descriptions were sent and pinned in Discord, and updated as needed. Additionally, as Discord was already the primary method of communication, new tasks were naturally being discussed in Discord when issues or missing features were encountered.

3. Implementation Methodology

The following sections describe the strategy used to develop the game.

3.1. Feature-Based

During development, the group used a feature based development strategy, where one or related features would be developed in isolation, and later integrated into the game. This approach was chosen, because it allowed for parallel development, and paired well with requirement definitions.

At the beginning of the process the game was defined through a set of user stories, each describing a unique feature. The user stories were largely independent from other user stories, or could be grouped together to form independent sets of features.

During development, these isolated features or sets would be used to distribute work among team members. As the user stories were categorized with MoSCoW, it was clear which features should be prioritized and developed first.

The prioritization was mostly compatible with this distribution of responsibilities, however, occasionally a feature would be blocked by another feature. In this case, a feature with lower prioritization might see development first, to make the most effective use of the time available.

3.2. Proof-of-Concept Development

For some features, experimental development or exploration was beneficial. Experiments were made as small proof-of-concept (PoC) implementations of a feature, where the team member responsible could freely investigate implementation strategies and experiment. This would help the group identify the best implementation strategies, or confirm whether a strategy was viable at all.

Some PoCs would show that the initial implementation strategy was indeed viable and well-suited for a feature, where others revealed issues, and a new strategy had to be tested. Some features that were explored in PoCs were scrapped. Some developer utilities were also tested in PoCs, such as split screen support.

3.3. Reflection on Implementation Methodology

Overall, the chosen development method was beneficial to the group. It was a familiar development methodology, that the group has used multiple times in other projects. This made the use of a feature based development strategy easy and natural. In addition, the project was defined through features, so feature based development could be applied easily.

It also enabled parallel implementation, where team members could work individually. This proved useful, as the group members have unique schedules, that don't always allow for cooperative work. Parallelism also integrates well with the group's Git workflow that will be described later.

4. Git Workflow

To manage collaboration and maintain a stable, qualitative code base during development, Parallax was version controlled on GitHub, with a feature-branch workflow. This allowed each member of the project to work independently on tasks while maintaining controlled integration to the main branch. This approach supported iterative development and allowed for minimal merge conflicts during implementation.

4.1. Repository setup

The project repository was hosted on GitHub. Direct commits to the `main` branch were restricted, and all development was conducted through feature branches to ensure stability.

Each project member had access to the Git repository and committed their changes to a feature branch[4] rather than directly to the `main` branch.

4.2. Branch strategy

Development followed a feature-branch strategy, depicted in **Figure 2**. For each task or feature, a separate branch was created from staging using descriptive names such as `player-movement` or `multiplayer-camera`. Once completed, the branch was merged into staging via pull requests after review by at least one team member.

Every task or feature was developed on their own branch, forked from the development branch, named “*staging*”. The naming convention for these feature branches was “**feat/<featureName>**”. Once a feature was completed, it was merged back into the *staging* branch.

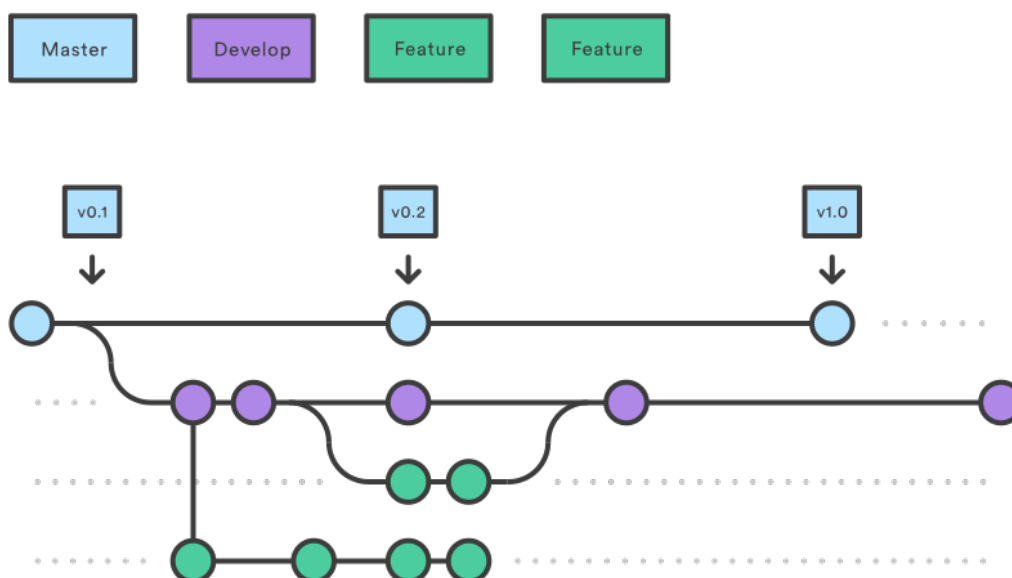


Figure 2: Feature-branching Git workflow [5]

4.3. Pull requests and code review

Whenever the staging branch contained a stable and working version of the game, a pull request was created to merge it back into the main branch. This ensured that main always contained a fully functional version of the game, from which a release could be created.

Code was to be reviewed by at least one other member of the team, before the pull request could be completed. If there were any comments on the code, it had to be reacted to, and the code would then be re-reviewed.

4.4. Conflict handling

Early development demonstrated that the most likely cause of conflicts, came from developers working on the same scene in Unity. An early strategic choice was made, to give each developer their own scene for them to work in. Whenever a feature was finished, it would be merged back to staging, and on staging, setup as intended for the feature in question.

While this mitigated most merge conflicts, the few that still remained were handled by the merging developer. This choice was made, based on the developer having the most in-depth knowledge of the merged code.

4.5. Commit practices

Commits were kept small and descriptive to ensure a good commit trail. Very small commits were sometimes necessary during collaborative coding, and did not follow any naming restrictions.

4.6. Reflection on Git workflow

Overall the feature branch strategy has worked very well for a project such as this. Early issues included incorrect .gitignore configuration, issues with developers using different editors and package manifests not being included in git. This was resolved very early and has not proven to be a problem since.

Using Git with Unity has it's issues, with .meta files and scenes, not being cooperative.

5. Discussion

The following sections highlight what worked well and worked didn't throughout the project, and propose what could have been done differently to mitigate some of the challenges encountered.

5.1. What Worked Well

The iterative development process allowed the group to improve the project through small steps and continuously adapt to changing requirements, rather than trying to finalize everything from the beginning. This was particularly advantageous, because the project goals were adjusted several times throughout the project.

The use of planning tools like Jira and Discord also worked well, because it allowed the group to keep track of tasks, and distribute responsibilities in a structured manner. Adjusting which tools were used during the project based on project needs also proved to be a good decision, because it allowed the group to work in the way that was most natural, making the process smooth and task tracking easy.

Frequent meetings and communication meant that the group was always informed about project progress, and gave group members good opportunities to seek help from each other if required. Good communication also ensured that the group always agreed on what the goals for the project were, and helped prevent the group from working towards diverging goals.

Distributing tasks among group members allowed the group to work more independently. This was beneficial because team members had unique personal schedules, that didn't always allow for cooperative work. It also made development more efficient, as multiple tasks were completed in parallel.

5.2. What Didn't Work Well

In the beginning of the project, certain goals were unclear. A lot of emphasis were put on deferring the graphical aspect of the game, so that more effort could be spent on the game mechanics and logical aspects. This led to some loss of focus in early development, as the group prioritized testing interactions between simple placeholder objects, temporarily losing sight of the intended cat-human theme of the game.

On some occasions, assumptions about the game or plans were made without full group agreement, leading to miscommunications and duplicated work. For example, whether meetings were held online or face-to-face were agreed on between meetings. On one occasion, such an agreement was made with one group member absent, leading to confusion about where the meeting was held, and unnecessary delays. This could have been prevented with clearer logistical planning, for example by using a planning tool to track agreed meeting times and locations.

Some features that were closely related or dependent on each other were developed in isolation to make more efficient use of time. This, however, led to some issues when integrating these features with each other later. On other occasions, a feature was finished, and later a new or changed requirement resulted in that feature requiring significant rework, leading to a lot of extra time spent on a feature that was completed. This could have been mitigated if interfaces and contracts between systems were very clearly defined.

5.3. Realizations

This project highlighted some key takeaways. Communication within the group must stay consistent and informative for the entire duration of the project. This will help prevent miscommunications and ensure that all group members are working towards the same goal. Additionally, many problems were solved faster in collaboration than alone.

The requirements became clearer after concrete development and experiments for the game were carried out. As mechanics and technologies were tested, the group was able to hone in on the intended game feel, and adapt the project goals to aim for the most enjoyable player experience possible.

Documentation also became more valuable as the project went on. The group routinely referred back to previously made documentation to confirm previous decisions or recall results of previous experiments.

Development methodologies like Scrum, feature based branching, and planning tools like Jira were a great help for structuring the development process. They assisted in tracking tasks, progress, and allowed the team to adapt along the way. Additionally, by adapting the tools and methodologies themselves to fit the team's working style made for a more comfortable and natural development process.

6. Conclusion

The project was carried out using an iterative development process. The workflow evolved over time based on the project's and group's needs. Tools and methodologies were adapted to fit the project. A Scrum inspired workflow was used, with development organized into 1-week sprints, which allowed the group to adapt. Jira and Discord were used for communication and task organization, giving a clear overview of tasks to be completed. Git was used for version control, and allowed the group to work on features in parallel.

The group used a feature based implementation strategy, which made it easy split work into isolated tasks, that could be distributed based on interest and competence. By creating small PoCs, the group was able to experiment with features and implementation strategies, and make informed decisions based on findings.

Throughout the project, the group held frequent meetings, and coordinated with each other over online communication channels. This helped ensure that everyone in the group was working towards the same goal. During the project, the group encountered occasional hiccups with coordination, which highlighted the importance of good communication.

Overall, the development process was smooth and efficient, apart from a few occasional issues. This has played a big role in getting to a stable, playable prototype, with nearly all planned features implemented.

Bibliography

- [1] "What is Scrum?." [Online]. Available: <https://www.scrum.org/resources/what-scrum-module>
- [2] "Jira." [Online]. Available: <https://www.atlassian.com/software/jira>
- [3] "Discord." [Online]. Available: <https://discord.com/>
- [4] P. Jain, "The Ultimate Guide to Git Branching Strategies." [Online]. Available: <https://blog.prateekjain.dev/the-ultimate-guide-to-git-branching-strategies-6324f1aceac2>
- [5] K. Patel, "Git Branching Strategies: A Comprehensive Guide." [Online]. Available: <https://dev.to/karmpatel/git-branching-strategies-a-comprehensive-guide-24kh#feature-branching>